Phase II Report

Pipelined RV64I Processor Design

Submitted by

Devang Bordoloi	Roll No: 2025122003	devang.bordoloi@research.iiit.ac.in
Tushar Gupta	Roll No: 2025122001	tushar.gupta@research.iiit.ac.in
Sai Ritvik Uppuganti	Roll No: 2025122012	sairitvik.uppuganti@research.iiit.ac.in

Department of Electronics and Communication Engineering

International Institute of Information Technology Hyderabad

March 10, 2026

Contents

Abstract

Phase II of our semester project successfully extends the previously implemented sequential RV64I processor into a fully pipelined 5-stage design. The pipeline overlaps execution of up to five instructions simultaneously, dramatically improving throughput over the single-cycle baseline. To maintain correctness, a centralized Hazard Unit provides full data forwarding, load-use stall insertion, and pipeline flushing for both taken branches and unconditional jumps. All modules are implemented in Verilog HDL, verified with Icarus Verilog, and waveforms inspected in GTKWave, demonstrating correctness across the full required RV64I instruction subset.

1 Introduction

Building upon the successfully implemented Sequential version of the RISC-V processor, Phase II of our semester project transitions the architecture to a 5-stage Pipelined Processor. While the sequential design was functional, it suffered from low instruction throughput as instructions were executed one at a time. This phase drastically improves instruction throughput by overlapping execution.

Our pipelined processor maintains full support for the same subset of the RISC-V ISA as the sequential design. The transition required decomposing the datapath into stages, establishing proper pipeline registers, and engineering comprehensive hazard detection and data forwarding mechanisms to maintain execution correctness.

2 Problem Statement and Requirements

Phase II extended the verified Phase I sequential processor with the following requirements:

- **Pipelined Execution:** Decompose the single-cycle datapath into a 5-stage pipeline (IF, ID, EX, MEM, WB) capable of executing multiple instructions simultaneously.
- **Instruction Support:** Maintain full support for the same instruction subset as Phase I: R-type (ADD, SUB, AND, OR), I-type arithmetic (ADDI, ANDI, ORI), load/store (LD, SD), and branch (BEQ).
- **Pipeline Register Design:** Implement four synchronous pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB) to latch data and propagate control signals stage-by-stage.
- **Data Hazard Resolution:** Implement a Forwarding Unit that resolves EX and MEM data hazards by bypassing results from later stages back to the ALU inputs without stalling.
- **Load-Use Hazard Detection:** Detect load-use hazards (an instruction directly following a LD reading the loaded value) and insert a one-cycle stall bubble.
- **Control Hazard Handling:** Use static not-taken branch prediction; flush the IF and ID stage contents on a taken branch or unconditional jump resolved in EX (both conditions are captured by the single `PCSrcE` signal).
- **Modular Design:** Retain the bottom-up philosophy with all datapath elements encapsulated in `data_unit_wrapper_final.v` and all control logic in `control_unit_wrapper.v`, wired by `pipelined_top_module.v`.
- **Verification:** Verify with Icarus Verilog and GTKWave; produce a final `register_file.txt` dump for automated grading.

3 The 5-Stage Pipeline Architecture

To achieve pipelining, the sequential datapath was divided into five distinct operational stages.

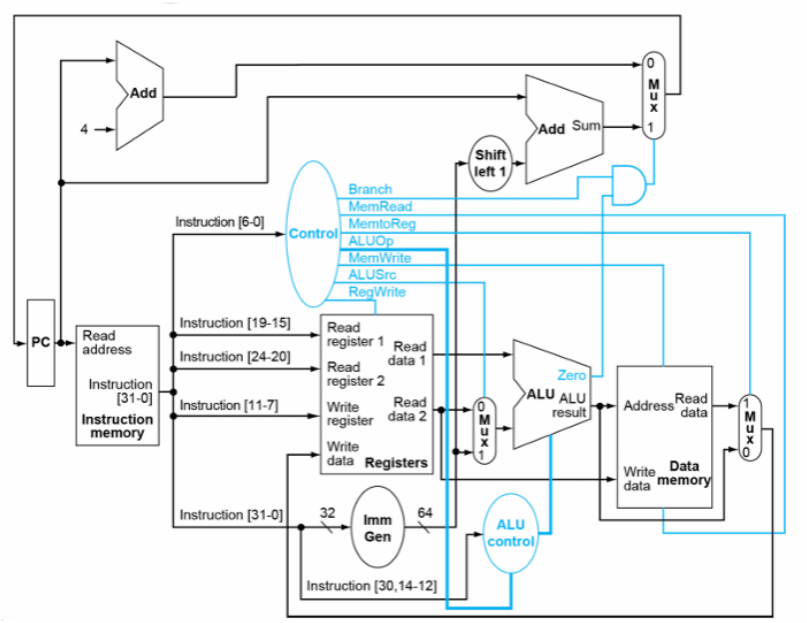

Figure 1: The 5-Stage Pipeline Architecture

- **Instruction Fetch (IF):** Fetches the instruction from Instruction Memory and increments the PC.
- **Instruction Decode (ID):** Decodes the instruction, reads from the Register File, and generates Immediate values.
- **Execute (EX):** Performs arithmetic operations (ALU) and calculates branch targets.
- **Memory (MEM):** Accesses Data Memory for load/store instructions.
- **Write Back (WB):** Writes results back to the Register File.

4 Reference Architecture

The structural design of our pipelined processor is heavily guided by standard pipeline hazard handling architectures, as illustrated in the reference datapath below.

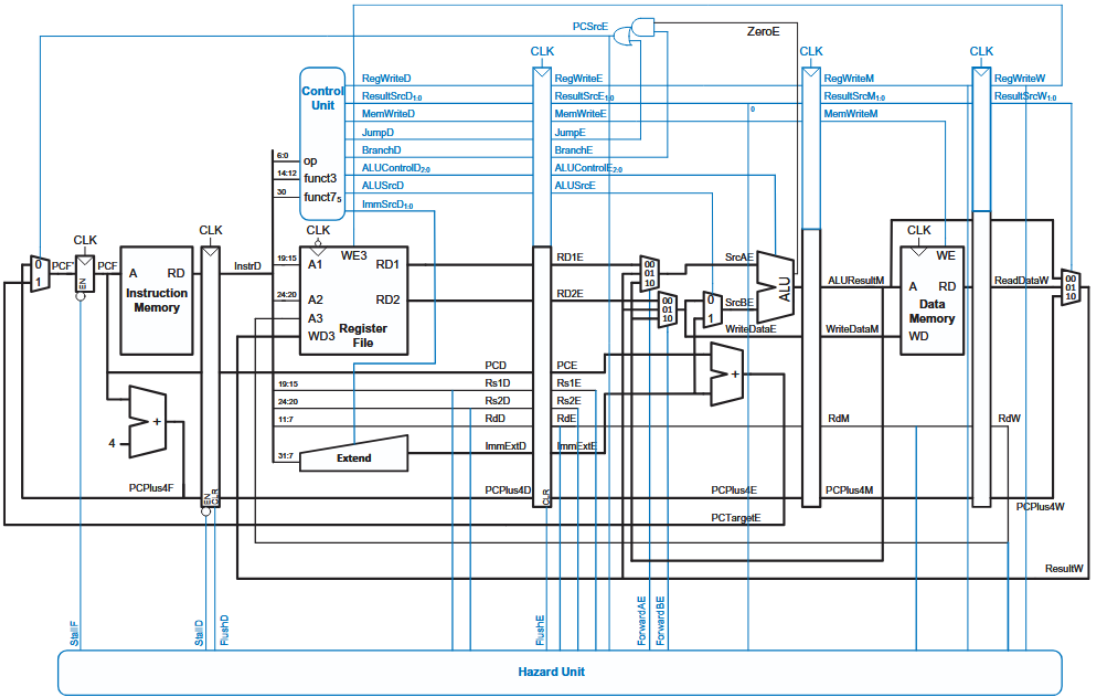

Figure 7.61 **Pipelined processor with full hazard handling**

Figure 2: Pipelined processor with full hazard handling (Figure 7.61)

This reference architecture (Figure 7.61) is implemented through several key hardware integrations across the datapath:

- **Stage Isolation:** The core datapath is divided by four pipeline registers that latch values on every rising clock edge: IF/ID (between Instruction Memory and the Register File), ID/EX (after the Register File and Extend unit), EX/MEM (after the ALU and branch adder), and MEM/WB (after Data Memory). Each stage boundary is explicitly clocked with its own `CLK` input.

- **Control Unit and Signal Propagation:** The Control Unit in the Decode stage decodes `op` (`instr[6:0]`), `funct3` (`instr[14:12]`), and `funct7[30]` and produces control signals: `RegWriteD`, $\texttt{ResultSrc}_{1:0}\texttt{D}$ (2-bit, encoding ALU result / memory read data / `PCPlus4` for the Write-Back mux), `MemWriteD`, `MemReadD`, `JumpD`, `BranchD`, $\texttt{ALUControl}_{3:0}\texttt{D}$, and `ALUSrcD`. Immediate format selection is handled internally by the Immediate Generator from the opcode; no separate `ImmSrc` signal is routed externally. These decoded signals are registered into the ID/EX pipeline register and propagated stage-by-stage as `*E`, `*M`, and `*W` variants so that each instruction carries its own control word through the pipeline.

- **PC and Result Buses:** Both the raw fetch address `PCF` and the incremented `PCPlus4F` are registered by the IF/ID stage, producing `PCD` and `PCPlus4D` respectively. `PCPlus4` continues propagating through ID/EX, EX/MEM, and MEM/WB (`PCPlus4E`, `PCPlus4M`, `PCPlus4W`) to support JAL link-address writes. The raw PC `PCE` (derived from `PCF` latched through IF/ID and ID/EX) is used in the EX stage to compute the branch/jump target `PCTargetE = PCE + ImmExtE` via a dedicated `adder_64` instance. The PC-redirect signal `PCSrcE` is formed in the EX datapath as the logical OR of a taken conditional branch (`BranchE && zero_flag_e`) and an unconditional jump (`JumpE`), and is fed back to the next-PC multiplexer to redirect fetch whenever either condition holds.

- **Execution Forwarding:** Each ALU operand is selected by a cascade of two 64-bit 2:1 multiplexers rather than a single 3:1 mux. For operand A: the first stage (textttmux_fwd_a_s1) selects between the Register File output `RD1E` and `ResultW` (WB-stage forwarding) using `ForwardAE[0]`; the second

stage (mux_fwd_a_s2) overrides with ALUResultM (MEM-stage forwarding) when ForwardAE[1] is asserted, producing SrcAE. An identical two-stage cascade produces WriteDataE for operand B, which also serves as the store data on SD instructions. A final ALUSrc mux then chooses between WriteDataE and ImmExtE to form SrcBE.

- **Write-Back Mux:** At the end of the pipeline the 2-bit ResultSrcW field (propagated through all four pipeline registers from ResultSrcD) drives two cascaded 2:1 multiplexers. The first stage selects between ALUResultW and ReadDataW using ResultSrcW[0] (asserted for load instructions). The second stage overrides with PCPlus4W using ResultSrcW[1] (asserted for jump instructions, writing the return address). The final output ResultW is written to the Register File and simultaneously fed back as the WB-stage forwarding source into the EX-stage forwarding multiplexers.
- **Centralized Hazard Unit:** A single purely combinatorial Hazard Unit, instantiated as a peer of the control unit and datapath wrapper at the top level, receives the source register fields from the ID-stage instruction word (rsD = InstrD[19:15], rtD = InstrD[24:20]) and the pipeline-registered EX-stage addresses rsE/rtE, together with WriteRegE/WriteRegM/WriteRegW and corresponding RegWrite enables. It also receives ResultSrcE[1:0] (to identify load instructions via ResultSrcE[0]) and the datapath-computed PCSrcE. From these inputs it drives ForwardAE/ForwardBE for EX-stage forwarding, StallF/StallD to freeze the PC and IF/ID register on a load-use hazard, and FlushD/FlushE to squash the ID and EX stages following a taken branch or jump.

5 Detailed Module Implementation

The pipelined processor reuses all low-level primitive and datapath modules from the Phase I sequential design without modification. These modules are reviewed briefly below for completeness before describing the new pipeline-specific components.

5.1 Low-Level Primitives and Hardware Math

The arithmetic foundation is carried over from Phase I unchanged. A 1-bit full_adder.v is chained via a generate block in 64bit_adder.v to form a 64-bit ripple-carry adder with carry-out and overflow detection. The 64-bit subtractor (64bit_subs.v) performs 2's complement subtraction by inverting the second operand through NOT gates and asserting carry-in to 1 through the same adder chain. Bitwise logical operations are implemented in 64bit_and.v, 64bit_or.v, and 64bit_xor.v using generate loops for scalable 64-bit operation.

5.2 Shift and Comparison Modules

Shift operations are implemented as combinatorial barrel shifters: sll_64.v (logical left), srl_64.v (logical right), and sra_64.v (arithmetic right). Each uses a cascade of 6 multiplexer stages to shift by 32, 16, 8, 4, 2, and 1 bit, controlled by the 6-bit shift amount. The arithmetic right shift explicitly concatenates the sign bit in place of zeros to preserve 2's complement negative values. Set-Less-Than comparisons (slt_64.v signed, sltu_64.v unsigned) route inputs through the subtractor and XOR the MSB of the difference with the overflow flag to handle signed overflow correctly. The 2:1 multiplexer is implemented in mux.v using a generate block that scales to 64-bit buses.

5.3 The Arithmetic Logic Unit (alu.v)

The alu_64_bit module instantiates all primitives above. A 4-bit alu_control opcode selects the correct result via an always @(*) case statement. The module exposes a zero_flag wire (used by the branch decision logic), as well as carry_flag and overflow_flag. In the pipeline, the ALU is located in the EX stage; its operand inputs SrcAE and SrcBE are driven by the forwarding multiplexers. The borrow flag for subtraction is generated by inverting the subtractor carry-out (carry_flag = !w_sub_cout).

5.4 Program Counter and Instruction Memory

- **Program Counter (pc.v):** A 64-bit synchronous register (textttprogram_counter) updating on every positive clock edge with synchronous reset to zero. StallF is a dedicated input port of the register module itself; when asserted, the sequential logic inside pc.v ignores the incoming

pc_in and holds its current value, freezing the fetch address for the stall cycle. The input pc_in is supplied by a 2:1 mux2_64 immediately upstream that selects between PCPlus4F (normal sequential advance) and PCTargetE (branch/jump target) according to PCSrcE.

- **PC Incrementer:** A separate adder_64 instance (pc_incrementer) adds the constant 64'd4 to PCF on every cycle to produce PCPlus4F. Its carry and overflow outputs are left unconnected as they are not meaningful for a byte-aligned PC.
- **Instruction Memory (instruction_mem.v):** A 4096-byte Big-Endian ROM. Four bytes are concatenated MSB-first to produce the 32-bit instruction InstrF, accessed combinatorially using the current PC value PCF.

5.5 Instruction Decode and Immediate Generation

- **Register File (reg_file.v):** A 32-entry 64-bit array with dual asynchronous read ports (Rs1D, Rs2D) and a single synchronous write port (RdW, ResultW, RegWriteW). Register x0 is hardwired to zero via a write guard (write_reg != 5'b0). In the pipeline, reads occur in ID while writes occur in WB—a two-stage separation that is the fundamental source of data hazards.
- **Immediate Generator (ig.v):** Uses the 7-bit opcode to select the instruction format and sign-extends the scattered immediate bits, using the Verilog replication operator {52{instr[31]}} to copy the sign bit for I-type, S-type, and B-type instructions. In the pipeline, the 64-bit sign-extended immediate imm_data is passed directly as ImmExtD into the ID/EX register, with no additional shifting applied in the ID stage.

5.6 Data Memory (data_mem.v)

The data memory models a byte-addressable Big-Endian RAM. It performs synchronous writes (MemWrite) and asynchronous reads (MemRead). On a 64-bit store, 8 bytes of write_data are stored MSB-first (memory[addr] <= write_data[63:56]). In the pipeline the address input is ALUResultM propagated from the EX/MEM register, and ReadDataM feeds the MEM/WB register.

5.7 Control Units (cu.v, alu_cu.v, control_unit_wrapper.v)

- **Main Control Unit (cu.v):** Decodes the 7-bit opcode with a case statement to assert RegWrite, MemRead, MemtoReg, MemWrite, ALUSrc, Branch, and a 2-bit ALUOp. All outputs default to 0 to prevent synthesis latches.
- **ALU Control (alu_cu.v):** Refines the 2-bit ALUOp with funct3 and funct7[30] to produce the 4-bit ALUControl opcode, distinguishing ADD/SUB, AND, OR, XOR, shifts, and set-less-than for both R-type and I-type instructions.
- **Control Unit Wrapper (control_unit_wrapper.v):** Instantiates three sub-modules: the main cu (Main Control Unit), the alu_cu (ALU Control), and a dedicated jump_control module (jal_jump.v) that asserts the Jump signal for JAL/JALR opcodes. The 2-bit ResultSrc output is derived by concatenating Jump and the internal w_MemtoReg wire: ResultSrc = {Jump, w_MemtoReg}. Critically, the branch decision (combining the branch flag with the ALU zero flag) is *not* performed here; it is resolved entirely within the EX stage of the datapath (and(br_taken_e, BranchE, zero_flag_e)). As a consequence, the zero_flag input port of the wrapper is tied to 1'b0 at the top level. In the pipeline, this wrapper is driven by InstrD field extractions (InstrD[6:0], InstrD[14:12], InstrD[30]) and drives all *D-suffixed control signals into the ID/EX register.

5.8 Pipeline Registers

To isolate the five stages and ensure data and control signals propagate synchronously, four pipeline registers were implemented. Control signals (e.g., RegWrite, MemRead, MemWrite, the 2-bit ResultSrc, Branch, Jump) propagate through these registers alongside the datapath values so that each instruction carries its own control word through the pipeline.

5.8.1 IF/ID Register (if_id.v)

This register forms the boundary between the Fetch and Decode stages.

- **Data Signals:** Captures two PC-related values and the fetched instruction. PCPlus4F (the already-incremented PC) is registered as PCPlus4D (IF_ID_pc_out) for continued propagation toward Write-Back. The raw fetch address PCF is separately registered as PCD and further forwarded by the ID/EX register as PCE, where it is used as the base for branch-target computation (PCTargetE = PCE + ImmExtE). The 32-bit fetched instruction InstrF is registered as InstrD (IF_ID_instr_out) for field extraction in the Decode stage.
- **Reset Behaviour:** On a synchronous reset, both the PC and instruction registers are driven to zero, placing the pipeline in a clean initial state.
- **Flush:** The flush input mirrors the reset behaviour—it forces both outputs to zero without requiring a full processor reset. This is asserted by the Hazard Unit whenever a taken branch is resolved in EX, discarding the incorrectly fetched instruction and inserting a bubble into the Decode stage.
- **Stall (Write Enable):** The active-high IF_ID_write signal acts as a clock enable. When held low by the Hazard Unit during a load-use hazard, the register retains its current PC and instruction values, effectively freezing the Fetch and Decode stages while the rest of the pipeline advances.
- **Priority:** The implementation enforces the priority order reset > flush > IF_ID_write, ensuring that reset and flush cannot be blocked by an active stall.

5.8.2 ID/EX Register (id_ex.v)

This register carries decoded data and control signals into the Execute stage.

- **Data Signals:** Buffers the 64-bit PC (ID_EX_pc_in), two 64-bit register read values (data_in_1, data_in_2), the 64-bit sign-extended immediate (imm_gen), and three 5-bit register addresses: the destination (ID_EX_rd_in) and both source registers (ID_EX_rs1_in, ID_EX_rs2_in).
- **Execute-Stage Control Signals:** Carries alu_src (selects between register value and immediate as the second ALU operand), the 4-bit alu_control encoding the ALU operation, and branch (flags the instruction as a branch for target computation).
- **Memory-Stage Control Signals:** Propagates mem_read and mem_write to govern Data Memory access in the subsequent stage.
- **Write-Back Control Signals:** Carries the 2-bit ResultSrc[1:0] (encoding ALU result, memory read data, or PCPlus4 for JAL link-address write-back), the Jump flag (propagated to EX to participate in PCSrcE generation via or(PCSrcE, JumpE, br_taken_e)), and reg_write_en (enables writing to the destination register). Additionally, both the decoded-stage PC (PCD → PCE) and PCPlus4D (→ PCPlus4E) are buffered so that the EX stage has access to both the instruction's own fetch address (for branch-target addition) and its return address (for JAL write-back).
- **Flush (Bubble Insertion):** On either reset or flush, every output—both data lines and all control signals—is driven to zero. This converts the in-flight instruction into a NOP bubble, which is essential when a stall cycle is inserted for a load-use hazard or when a mispredicted branch must be squashed.
- **Source Register Forwarding Support:** By retaining rs1 and rs2 addresses through to the EX stage, the Forwarding Unit can compare them against destination registers in later stages and assert the correct ForwardAE/ForwardBE mux selects.

5.8.3 EX/MEM Register (ex_mem.v)

This register isolates the Execute and Memory stages.

- **Data Signals:** Latches the 64-bit alu_out (the computed address or arithmetic result) and data2 (the value to be written on a store instruction). It also retains the 5-bit destination register address rd and the 5-bit source register rs2_ID_EX.

- **Memory-Stage Control Signals:** Propagates `mem_read` to enable a load from Data Memory and `mem_write` to enable a store to Data Memory.
- **Write-Back Control Signals:** Carries the 2-bit `ResultSrcE` (registered as `ResultSrcM` at the output) and `reg_write_en` forward to the MEM/WB register so the Write-Back stage can select the correct result and finalize the register write. Additionally, `PCPlus4E` is registered as `PCPlus4M`, continuing the propagation of the JAL link address toward Write-Back.
- `rs2_ID_EX` **Forwarding Support:** The original RS2 address is preserved alongside `data2`. While not actively consumed by the current Hazard Unit for additional forwarding, it is retained for potential store-data forwarding extensions.
- **Reset Behaviour:** A synchronous `reset` drives all outputs to zero. Unlike IF/ID and ID/EX, this register has no `flush` input—by the time an instruction reaches this stage, branch resolution has already occurred in EX and any necessary flushes have been applied to earlier stages.

5.8.4 MEM/WB Register (`mem_wb.v`)

The final pipeline register sits between the Memory and Write Back stages.

- **Data Signals:** Buffers three values from the Memory stage: the 64-bit `data` read from Data Memory (used by load instructions), the 64-bit `alu_out` passed through from EX/MEM (used by R-type and I-type instructions), and the 5-bit destination register address `rd` that identifies where the result is written.
- **Write-Back Selection (`ResultSrcW`):** The 2-bit `ResultSrcW` field (propagated from `ResultSrcM`) drives two cascaded 2:1 muxes in the Write-Back stage. `ResultSrcW[0]` selects between `ALUResultW` (for R-type/I-type) and `ReadDataW` (for load instructions). `ResultSrcW[1]` then overrides with `PCPlus4W` for jump instructions, writing the return address. The register also carries `PCPlus4W` (registered from `PCPlus4M`) specifically for this purpose.
- **Register Write Enable (`reg_write_en`):** Forwarded to both the Register File write-enable port and to the Hazard Unit's forwarding logic (`RegWriteW`), which checks it to confirm that a result in the WB stage is valid before applying WB-to-EX forwarding.
- **Minimal Interface:** Because this is the last pipeline register, no Execute- or Memory-stage control signals need to propagate further; only `ResultSrcW`, `reg_write_en`, `PCPlus4W`, `alu_out`, `data`, and `rd` are retained, keeping the hardware interface lean.
- **Reset Behaviour:** A synchronous `reset` sets all outputs to zero. No `flush` input is present, as instructions in the WB stage have already completed all potentially hazardous operations.

5.9 Hazard Handling Unit (`hazard_unit.v`)

The Hazard Unit is a purely combinatorial module that monitors register addresses across pipeline stages and drives stall, flush, and forwarding control signals. It is the component that makes the pipeline execution-correct in the presence of data and control dependencies.

5.9.1 Data Forwarding

In the EX stage, each ALU operand is selected by a cascade of two 64-bit 2:1 multiplexers (`mux2_64`) controlled by the 2-bit `ForwardAE` and `ForwardBE` signals from the Hazard Unit. The Hazard Unit uses two independent `always@(*)` blocks—one for each forward signal—implementing the same priority-encoded if/else-if/else chain. MEM-stage forwarding (`2'b10`) is checked first; WB-stage forwarding (`2'b01`) is the fallback; and `2'b00` (no forwarding) is the default. Because MEM is checked before WB, a RAW hazard that matches both stages (possible when the same register is written twice in consecutive instructions) correctly forwards the most recent result. The Hazard Unit computes these as follows:

- **EX Hazard (MEM → EX):** If `rsE != 0` and `rsE == WriteRegM` and `RegWriteM` is high, `ForwardAE` is set to `2'b10`. The forwarding mux cascade selects `ALUResultM` as `SrcAE`, bypassing the stale Register File value. The same comparison is made for `rtE` to compute `ForwardBE`.

- **MEM Hazard (WB → EX):** If the MEM condition does not hold but `rsE != 0` and `rsE == WriteRegW` and `RegWriteW` is high, `ForwardAE` is set to `2'b01`, selecting `ResultW` (the final Write-Back result looped back from the WB mux output).
- **No Hazard:** Forward signal is `2'b00`, selecting the Register File output directly.
- **Safety Constraints:** Forwarding is disabled if the destination register is `x0` (hardwired zero) or the corresponding `RegWrite` is not asserted.

5.9.2 Load-Use Hazard Detection and Stall

Forwarding alone cannot resolve a load-use hazard: when a `LD` is in EX, the loaded value is not available until the end of MEM—one cycle too late for a directly following dependent instruction. Load instructions are identified by inspecting `ResultSrcE[0]`, the low bit of the 2-bit `ResultSrc` field, which is asserted whenever the first Write-Back mux stage selects memory read data (i.e., the instruction is a load). The Hazard Unit detects a load-use stall as:

$$\texttt{lwstall} = \texttt{ResultSrcE[0]} \ \wedge \ (\texttt{rsD} = \texttt{WriteRegE} \ \vee \ \texttt{rtD} = \texttt{WriteRegE})$$

When `lwstall` is asserted:

- `StallF` freezes the Program Counter (PC retains its value next cycle).
- `StallD` disables the IF/ID register write-enable, freezing the fetched instruction in Decode.
- `FlushE` clears all ID/EX outputs to zero, inserting a NOP bubble into the Execute stage.

5.9.3 Control Hazard Handling

The processor uses a static **not-taken** branch prediction strategy. It continues fetching instructions sequentially (`PC+4`) while the branch is evaluated in EX.

- `PCSrcE` is the combined redirect signal computed in the EX datapath as the logical OR of a taken branch and an unconditional jump. In the Verilog, this is implemented with two primitive gate instantiations: `and(br_taken_e, BranchE, zero_flag_e)` produces the taken-branch wire, and `or(PCSrcE, JumpE, br_taken_e)` ORs it with the Jump flag. `PCSrcE` is therefore a purely combinatorial wire driven from EX-stage register outputs, not a registered flip-flop. It is fed back to the next-PC mux and forwarded to the Hazard Unit simultaneously.
- On `PCSrcE` assertion: `FlushD` clears the IF/ID register (squashing the instruction in Decode) and `FlushE` clears the ID/EX register (squashing the instruction in Execute). This incurs a fixed 2-cycle penalty for any taken branch or jump.
- On a not-taken branch: `PCSrcE` remains low, no flush occurs, and execution continues without penalty.
- The branch/jump target `PCTargetE = PCE + ImmExtE` is computed in EX by a dedicated `adder_64` instance. The immediate `ImmExtE` is the raw sign-extended value from the Immediate Generator; the B-type encoding in `ig.v` already places bit 0 as `1'b0`, so the adder directly produces a correctly half-word-aligned target address.

5.10 Top-Level Integration (`data_unit_wrapper_final.v`, `pipelined_top_module.v`)

The `pipelined_data_unit` module in `data_unit_wrapper_final.v` instantiates every datapath component in pipeline-stage order: a next-PC 2:1 mux and `program_counter` (with `StallF`-gating) in IF; `instruction_mem` and the IF/ID register (capturing both `PCF` and `PCPlus4F`); `reg_file`, `ig`, followed by the ID/EX register in ID; the two-stage forwarding-mux cascades, `alu_64_bit`, a dedicated `adder_64` for branch-target computation, and the EX-stage `PCSrcE` logic (`and`/`or` gates) followed by the EX/MEM register in EX; `data_mem` and the MEM/WB register in MEM; and two cascaded 2:1 Write-Back muxes driven by `ResultSrcW` in WB.

The Hazard Unit (`hazard_unit.v`) is intentionally *not* embedded in the data wrapper. Instead, the top-level module `risc_v_top` (`pipelined_top_module.v`) coordinates three peer instantiations: `control_unit_top`,

`hazard_unit`, and `pipelined_data_unit`. This three-way structure cleanly separates concerns: the control unit decodes the instruction word and generates `*D`-suffixed signals; the Hazard Unit monitors pipeline-register addresses exposed by the datapath and drives stall, flush, and forwarding controls back into it; and the datapath executes instructions, exposing `RsE`, `RtE`, `RdE/RdM/RdW`, `RegWriteE/M/W`, `ResultSrcE`, and `PCSrcE` as top-level outputs for the Hazard Unit to observe.

6 Verification and Simulation Results

Our verification followed the same test-driven methodology as Phase I, extended to cover pipeline-specific correctness including forwarding paths and stall/flush behaviour.

6.1 Simulation Methodology

1. **Unit Testing:** Individual modules (ALU, Register File, Data Memory, Hazard Unit) were tested with dedicated testbenches exercising corner cases: x0 forwarding suppression, simultaneous stall and flush, and Big-Endian byte ordering.

2. **Integration Testing:** The top-level testbench `pipe_tb.v` loads `instructions.txt` into instruction memory and clocks the pipeline. The testbench dynamically halts when `InstrF` becomes all-zeros or undefined (program end), avoiding any fixed-cycle timeout.

3. **VCD Generation:** The testbench calls `$dumpfile("pipe_processor.vcd")` and `$dumpvars(0, pipe_tb)` to capture a full signal dump of every wire and register for GTKWave analysis.

4. **Automated Output:** The Register File writes its final 32-register state to `register_file.txt` using `$fdisplay` upon simulation completion, enabling automated grading.

6.2 Pipeline Execution Waveform

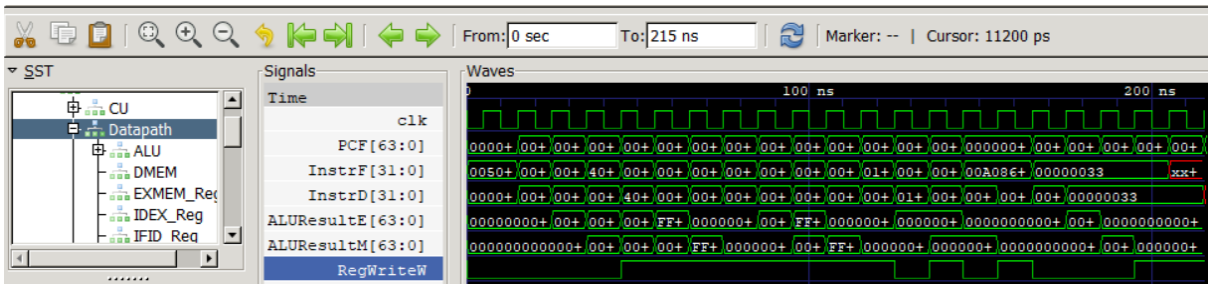

Figure 3: GTKWave snapshot of the pipelined processor executing the test program

The waveform confirms correct multi-stage overlap: at any given clock edge up to five different instructions are in distinct pipeline stages simultaneously. The PC increments by 4 on every non-stalled cycle. Stage-wise values (`PCF`, `PCD`, `PCE`, `ALUResultE`, `ALUResultM`) can be traced across consecutive cycles to verify that each instruction advances through the pipeline correctly.

6.3 Forwarding and Hazard Verification

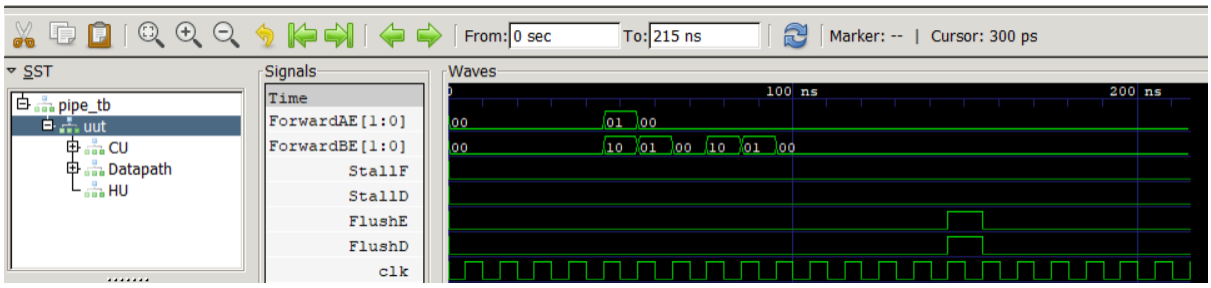

Figure 4: GTKWave snapshot showing data forwarding and branch flush signals from `hazard_inst`

- **EX/MEM Forwarding (`ForwardBE = 2'b10`):** The waveform clearly shows `ForwardBE` cycling through `10` and `01` on consecutive clock cycles, confirming that the Hazard Unit correctly detects back-to-back data dependencies and selects `ALUResultM` (MEM-stage result) as the forwarded operand into the ALU's B-input without inserting any stall.
- **WB Forwarding (`ForwardAE/ForwardBE = 2'b01`):** `ForwardAE` toggles to `01` when a result two cycles earlier (in WB) is needed by the current EX instruction, selecting `ResultW` from the Write-Back stage.
- **Branch Flush:** Near the end of the simulation (150 ns), `FlushE` and `FlushD` both rise high simultaneously for exactly one cycle, confirming that the taken BEQ causes both the ID/EX and IF/ID registers to be cleared in the same clock cycle, correctly squashing the two mis-fetched instructions.
- **No Load-Use Stalls:** `StallF` and `StallD` remain at zero throughout the entire execution, indicating that no load-use hazard occurs in the `instructions.txt` test program (no `LD` is immediately followed by a dependent instruction).

6.4 Summary of Verified Instruction Support

Table 1 summarises the instruction classes verified through simulation on the provided `instructions.txt` test program.

Table 1: Summary of Verified Instruction Support

Instruction	Type	Verified via Simulation
ADD	R-type	Yes
SUB	R-type	Yes
AND	R-type	Yes
OR	R-type	Yes
ADDI	I-type	Yes
LD	I-type (Load)	Yes
SD	S-type (Store)	Yes
BEQ	B-type	Yes

6.5 Final Register State

The Register File module outputs `register_file.txt` upon simulation completion via `$fdisplay`. This file contains the hexadecimal values of all 32 registers (x0–x31) and was cross-checked against manually traced expected results for the `instructions.txt` test program to confirm correct Write Back operation across all instruction types. The last line records the total clock cycles taken. Below is the generated output:

```
x0  = 0000000000000000
x1  = 000000000000000f
x2  = fffffffffffffffb
x3  = 000000000000000a
x4  = 000000000000000a
x5  = 000000000000000a
x6  = fffffffffffffffb
x7  = fffffffffffffffb
x8  = 0000000000000000
x9  = 0000000000000000
x10 = 000000000000000f
x11 = fffffffffffffffb
x12 = 0000000000000000
x13 = 000000000000001e
x14 = 0000000000000000
x15 = 0000000000000000
x16 = 0000000000000000
x17 = 0000000000000000
x18 = 0000000000000000
x19 = 0000000000000000
x20 = 0000000000000000
x21 = 0000000000000000
x22 = 0000000000000000
x23 = 0000000000000000
x24 = 0000000000000000
x25 = 0000000000000000
x26 = 0000000000000000
x27 = 0000000000000000
x28 = 0000000000000000
x29 = 0000000000000000
x30 = 0000000000000000
x31 = 0000000000000000

Total clock cycles: 21
```

Listing 1: register_file.txt – final register state after simulation

7 Challenges Encountered

7.1 Hazard Interaction Complexity

The most significant challenge was ensuring stall and flush signals did not interfere with each other when a load-use hazard and a taken branch occurred in close proximity. The implementation correctly handles the case by asserting `FlushE = lwstall | PCSrcE` (flush for either reason) while `FlushD` is raised only for the branch, and `StallF`/`StallD` only for the load stall. This required careful signal tracing across several test cases to verify no interactions were missed.

7.2 Control Signal Propagation Across Stages

Ensuring every control signal (`MemRead`, `MemWrite`, `RegWrite`, the 2-bit `ResultSrc`, `ALUSrc`, `ALUControl`, `Branch`, `Jump`) was correctly wired into the right pipeline register and propagated to the right stage required meticulous inter-module wiring. The replacement of the single-bit `MemtoReg` with the 2-bit `ResultSrc` field (to accommodate JAL write-back alongside load and ALU paths) was a particular source of careful reworking. A dropped signal in the ID/EX register caused silent corruption (e.g., spurious memory writes on non-store instructions) that only manifested mid-program.

7.3 Big-Endian Memory Carry-Over

The Big-Endian byte-ordering constraint inherited from Phase I remained a subtle source of errors during pipeline integration. Ensuring that both the instruction fetch and the data memory store/load oper-

ations maintained consistent byte ordering required careful inspection of GTKWave memory interface waveforms.

7.4 Dynamic Simulation Termination

As in Phase I, a fixed-cycle timeout was initially used, causing incorrect cycle counts and occasional premature termination before Write Back of the last instruction completed. The testbench was updated to monitor `InstrF` for an all-zero or undefined fetch as the halt condition, matching the Phase I approach. To complement this, the test program is padded with a sequence of NOP (all-zero) instructions at the end, ensuring the pipeline drains completely before the halt condition is triggered. This padding guarantees that the final real instruction has propagated through all five stages and been written back before the simulation terminates, giving an accurate total clock cycle count.

8 Conclusion

Phase II has been successfully completed, delivering a fully functional 5-stage pipelined RV64I processor that meets all specified requirements. The pipeline achieves correct concurrent execution of up to five instructions through rigorous hazard management: full data forwarding eliminates stalls for EX and MEM data hazards, a load-use detection circuit (keyed on `ResultSrcE[0]`) inserts a single NOP bubble for load-dependent instructions, and a flush mechanism driven by `PCSrcE` corrects the two-instruction penalty for taken branches and unconditional jumps. The modular design philosophy from Phase I was preserved and extended with a clean three-way separation between the control wrapper (`control_unit_wrapper.v`), the Hazard Unit (`hazard_unit.v`), and the datapath wrapper (`data_unit_wrapper_final.v`), all coordinated at the top level by `risc_v_top` in `pipelined_top_module.v`. This pipelined processor represents a substantial performance improvement over the sequential baseline and establishes a solid foundation for future microarchitectural enhancements.

Team Member Contributions

Tushar Gupta
Tushar was responsible for designing and implementing the Hazard Detection. He also built the top-level wrapper file to integrate all pipeline stages along with Ritvik. He also performed system-level testing to verify hazard behavior, helped debug the pipeline alongside Devang, and fixed previously developed modules to work with the pipelined datapath.

Sai Ritvik Uppuganti
Ritwik authored the technical report. He made top-level wrapper, data wrapper and control wrapper file along with Tushar by reviewing signal connectivity across pipeline stages. He participated in integration testing and helped fix issues in previously submitted modules.

Devang Bordoloi
Devang designed and implemented all four pipeline stage registers (IF/ID, ID/EX, EX/MEM, MEM/WB) with stall and flush control support. He developed the simulation testbench pipe_tb.v. And he collaborated with Tushar on debugging pipeline register timing and flush issues.He also made the report along with Ritvik.

9 References

1. Patterson, D. A., & Hennessy, J. L. (2017). *Computer Organization and Design RISC-V Edition: The Hardware Software Interface.* Morgan Kaufmann.
2. RISC-V International. (2019). *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA.*
3. Harris, S., & Harris, D. (2021). *Digital Design and Computer Architecture: RISC-V Edition.* Morgan Kaufmann.